

Reviewer Pack — Minimal Brauer-Induced Tensor Product Rank Correlation with ...

Entry ae801b5656f0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 07:27:18 UTC

Minimal Brauer-Induced Tensor Product Rank Correlation with Resolution Proof Width

Entry ID: ae801b5656f0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 07:27:18 UTC

1. Conjecture

Field A (mathematical branch): Brauer Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G , the minimal rank of the tensor product induced by its associated Brauer group modulo p , denoted as $BPrank(\varphi_G)$, is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $BPrank(\varphi_G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

Brauer theory provides a framework to study linear algebraic structures in noncommutative rings. The tensor product rank of Brauer classes has not been extensively applied in complexity theory, offering potential for new insights into the structure of resolution proofs.

Taxonomy category: brauer_theory_resolution_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3f358129f0f843ed

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Tseitin formula φ_G , if the Pearson correlation coefficient between $BPrank(\varphi_G)$ and $w(\varphi_G)$ is ≥ 0.8 AND the p-value is ≤ 0.05 for 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Brauer Theory" AND "resolution proof complexity" - "tensor product rank correlation" AND Brauer group - "minimal rank Tseitin formula" AND resolution proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_tseitin_formula(n):
    if n < 3:
        return None

    variables = [f'x{i+1}' for i in range(n)]
    clauses = []

    # Generate clauses for the Tseitin formula
    for i in range(1, n-1):
        clauses.append(f'({variables[i-1]} ^ {variables[i]}) v ~{variables[i+1]}')

    # Add final clause
    clauses.append(f'{variables[-2]} v ~{variables[-1]}')

    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    metric_values = []

    for n in range(5, n_max + 1):
        formula = generate_tseitin_formula(n)
        if formula is None:
            continue

    # Simulate Brauer group and tensor product rank (placeholder values)
    BPrank = random.randint(1, n)
```

```

# Simulate resolution proof width (placeholder values)
w = random.randint(10, 2 * n)

metric_values.append(BPrank / w)

if not metric_values:
    return {
        "metric_name": "BPrank/w",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean = sum(metric_values) / len(metric_values)
variance = sum((x - mean) ** 2 for x in metric_values) / len(metric_values)
std_dev = math.sqrt(variance)

return {
    "metric_name": "BPrank/w",
    "metric_value": mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
undefined'}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.4910278082675323, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.43665094980972863, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.635897293465028, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5653920435670093, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5020724020331974, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5439531738058201, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5783647315324868, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5222047251587447, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5121706099427268, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.5632452813276935, 'instances_tested': 30, 'n_max': 100}  
TRIAL: {'metric_name': 'BPrank/w', 'metric_value': 0.3353833475691384, 'instances_tested': 30, 'n_max': 100}  
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder values for the Brauer group and tensor product rank, which do not correspond to actual computations of these mathematical objects. The conjecture states a correlation between BPrank and resolution proof width, but the test does not compute either quantity correctly.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses placeholder values for the Brauer group and tensor product rank, which do not correspond to actual computations of these mathematic | next: Re-implement the test code to correctly compute BPrank and resolution proof width before re-evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17253
2	propose	ollama_remote	glm4:latest	0	0	13506
3	propose	ollama_remote	glm4:latest	0	0	12520
4	preregistration	ollama_remote	glm4:latest	0	0	9711
5	novelty	ollama_remote	glm4:latest	0	0	12476
6	novelty	ollama_remote	glm4:latest	0	0	9901
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25633
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23395
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15497
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10507
11	critic	ollama_remote	glm4:latest	0	0	14251
12	judge	ollama_remote	glm4:latest	0	0	17211

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 181860 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/ae801b5656f0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ae801b5656f0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ae801b5656f0.tar.gz (if generated)